TOKEN REPLACEMENT IN ASP.NET CORE WEB API

What is Token Replacement?

Tokens are **placeholders** enclosed in square brackets [] used within route templates. At runtime, ASP.NET Core automatically replaces these tokens with actual names.

Token	Replaced with	Example
[controller]	Controller name (without "Controller" suffix)	BooksController → Books
[action]	Action method name	GetAllBooks → GetAllBooks

Why Use Token Replacement?

<u>Benefit</u>		<u>Explanation</u>
No Hardcoding	→	Don't write controller/action names in routes.
Auto-Updates	→	Rename controller → Routes update Automatically.
Consistent	→	All methods follow same URL pattern
DRY Principle	→	Define pattern once, apply to all methods

Real-time Analogy: Tokens ***[controller]*** and ***[action]*** are replaced at runtime.

Now we are going to set up the project,

Project Setup

<u>Create the project:</u>

Command: dotnet new webapi -n LibraryApi3 -f net8.0

<u>Navigate to the project:</u>

Command: cd LibraryApi3

Install Packages

Before installing the packages, we should make sure that we are in the project folder 'LibraryApi3' in terminal. If not, then navigate to the exact project folder in terminal.

Command 1: dotnet add package Microsoft.EntityFrameworkCore --version 8.0.0

Command 2: dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0

Command 3: dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.0

Command 4: dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.0

To check, whether the packages are successfully set up or not, we have to look over .csproj file,

```
LibraryApi3.csproj
LibraryApi3.csproj
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="8.0.24" />
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>

</Project>
```

Step 1: Create Model

Path: LibraryApi3/Models/Book.cs

```
Book.cs
Models > Book.cs > ...
namespace LibraryApi3.Models
{
    0 references
    public class Book
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Title { get; set; } = string.Empty;
        0 references
        public string Author { get; set; } = string.Empty;
        0 references
        public string Genre { get; set; } = string.Empty;
        0 references
        public int PublishedYear { get; set; }
        0 references
        public bool IsAvailable { get; set; }
        0 references
        public decimal Price { get; set; }
    }
}
```

Step 2: Create In-Memory Data (7 Records)

Path: LibraryApi3/Data/BookData.cs

```
BookData.cs ×
Data > BookData.cs > ...
using LibraryApi3.Models;

namespace LibraryApi3.Data
{
    0 references
    public static class BookData
    {
        0 references
        public static List<Book> Books = new List<Book>
        {
            new Book {
                Id = 1,
                Title = "The Hobbit",
                Author = "J.R.R. Tolkien",
                Genre = "Fantasy",
                PublishedYear = 1937,
                IsAvailable = true,
                Price = 15.99m
            },
            new Book {
                Id = 2,
                Title = "1984",
                Author = "George Orwell",
                Genre = "Dystopian",
                PublishedYear = 1949,
                IsAvailable = false,
                Price = 12.99m
            },
            new Book {
                Id = 3,
                Title = "Pride and Prejudice",
                Author = "Jane Austen",
                Genre = "Romance",
                PublishedYear = 1813,
                IsAvailable = true,
                Price = 9.99m
            },
            new Book {
                Id = 4,
                Title = "The Great Gatsby",
                Author = "F. Scott Fitzgerald",
                Genre = "Classic",
                PublishedYear = 1925,
                IsAvailable = true,
                Price = 11.99m
            },
            new Book {
                Id = 5,
                Title = "Harry Potter",
                Author = "J.K. Rowling",
                Genre = "Fantasy",
                PublishedYear = 1997,
                IsAvailable = true,
                Price = 19.99m
            },
            new Book {
                Id = 6,
                Title = "Moby Dick",
                Author = "Herman Melville",
                Genre = "Adventure",
                PublishedYear = 1851,
                IsAvailable = false,
                Price = 14.99m
            },
            new Book {
                Id = 7,
                Title = "The Shining",
                Author = "Stephen King",
                Genre = "Horror",
                PublishedYear = 1977,
                IsAvailable = true,
                Price = 13.99m
            }
        };
    }
}
```

We are using this class for our api project to avoids using database for now, because SQL is not mandatory for this current project.

Step 3: Program.cs (Minimal Setup)

Path: LibraryApi3/Program.cs

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Step 4: Controller WITHOUT Token Replacement (For Comparison)

Path: LibraryApi3/Controllers/BookController.cs

BooksController.cs

Controllers > BooksController.cs > ...

```
using LibraryApi3.Data;
using LibraryApi3.Models;
using Microsoft.AspNetCore.Mvc;

namespace LibraryApi3.Controllers
{
    [ApiController]
    0 references
    public class BooksController : ControllerBase
    {
        // PROBLEM: Hardcoded routes - if controller name changes,
        // routes break!
        [Route("Books/GetAllBooks")]
        [HttpGet]
        0 references
        public ActionResult<IEnumerable<Book>> GetAllBooks()
        {
            return Ok(BookData.Books);
        }

        [Route("Books/GetAvailableBooks")]
        [HttpGet]
        0 references
        public ActionResult<IEnumerable<Book>> GetAvailableBooks()
        {
            var books = BookData.Books.Where(b => b.IsAvailable).ToList();
            return Ok(books);
        }

        [Route("Books/GetBookById/{id}")]
        [HttpGet]
        0 references
        public ActionResult<IEnumerable<Book>> GetBooksById(int id)
        {
            var book = BookData.Books.FirstOrDefault(b => b.Id == id);
            if (book == null)
            {
                return NotFound();
            }
            return Ok(book);
        }
    }
}
```

Drawback: If we rename ***BooksController*** to ***LibraryController***, all routes (Books/…) break and need manual updates.

Step 5: Controller WITH Token Replacement (The Main Focus)

Path: LibraryApi3/Controllers/LibraryController.cs

LibraryController.cs

Controllers > LibraryController.cs > { } LibraryApi3.Controllers > LibraryController > GetAvailableB

```
using LibraryApi3.Data;
using LibraryApi3.Models;
using Microsoft.AspNetCore.Mvc;

namespace LibraryApi3.Controllers
{
    [ApiController]

    // [controller] is replaced with "Library" (class
    // name without "Controller")
    [Route("controller")]
    0 references
    public class LibraryController : ControllerBase
    {
        // ==================================================
        // METHOD 1: Token at Controller level only
        // URL becomes: /Library/GetAllBooks
        // ==================================================
        [Route("[action]")]
        [HttpGet]
        0 references
        public ActionResult<IEnumerable<Book>> GetAllBooks()
        {
            return Ok(BookData.Books);
        }

        // ==================================================
        // METHOD 2: Combined token at Controller level
        // URL becomes: /Library/GetAvailableBooks
        // ==================================================
        [Route("[action]")]
        [HttpGet]
        0 references
        public ActionResult<IEnumerable<Book>> GetAvailableBooks()
        {
            var books = BookData.Books.Where(b => b.IsAvailable).ToList();
            return Ok(books);
        }

        // ==================================================
        // METHOD 3: Token + Route Parameter
        // URL becomes: /Library/GetBookById/3
        // ==================================================
        [Route("[action]/{id}")]
        [HttpGet]
        0 references
        public ActionResult<Book> GetBookById(int id)
        {
            var book = BookData.Books.FirstOrDefault(b => b.Id == id);
            if (book == null)
            {
                return NotFound($"Book with ID {id} not found");
            }
            return Ok(book);
        }
    }
}
```

What Happens at Runtime:

Token	Replaced With
[controller]	**Library**
[action] in **GetAllBooks**	**GetAllBooks**
[action] in **GetAvailableBooks**	**GetAvailableBooks**
[action] in **GetBookById**	**GetBookById**

Generated URLs like:

- */Library/GetAllBooks*
- */Library/GetAvailableBooks*
- */Library/GetBookById/3*

Adavntages:

Rename ***LibraryController*** to ***BookController*** → URLs automatically become */Book/GetAllBooks*, */Books/GetAvailableBooks*, */Book/GetBookById/3* – no code changes needed!

Step 6: Alternative – Token at Controller Level (Cleaner Approach)

Path: LibraryApi3/Controllers/StaffController.cs

StaffConroller.cs

Controllers > StaffConroller.cs > ...

```
using LibraryApi3.Data;
using LibraryApi3.Models;
using Microsoft.AspNetCore.Mvc;

namespace LibraryApi3.Controllers
{
    [ApiController]

    // define pattern ONCE at controller level - applices to ALL actions
    [Route("[controller]/[action]")]
    0 references
    public class StaffController : ControllerBase
    {
        // No [ROute] attributes nedded here - inherits from controller
        [HttpGet]
        0 references
        public ActionResult<IEnumerable<Book>> GetAllBooks()
        {
            return Ok(BookData.Books);
        }

        // Inherits same automatically
        [HttpGet]
        0 references
        public ActionResult<IEnumerable<Book>> GetAvailableBooks()
        {
            var books = BookData.Books.Where(b => b.IsAvailable).ToList();
            return Ok(books);
        }

        // Can add parameters - pattern becomes /staff/GetBookById/3
        [HttpGet]
        0 references
        public ActionResult<Book> GetBookById(int id)
        {
            var book = BookData.Books.FirstOrDefault(b => b.Id == id);
            if (book == null)
            {
                return NotFound();
            }

            return Ok(book);
        }
    }
}
```

Generated URLs:

- /Staff/GetAllBooks
- /Staff/GetAvailableBooks
- /Staff/GetBookById/3

This is the cleanest approach! Define pattern once at top, all methods automatically follow it.

Test our API – All Endpoints with Expected Responses

Let's build and run the application. And here we will use the command line or cli. We have to navigate to the exact project folder (LibraryApi3) and run those following commands,

Build the application,

Command: *dotnet build*

```
PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi3> dotnet build
  Determining projects to restore...
  All projects are up-to-date for restore.
  LibraryApi3 -> E:\PracticeProjects\DotnetApi\Routing\LibraryApi3\bin\Debug\net8.0\LibraryApi
  3.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:02.34
```

To run the application,

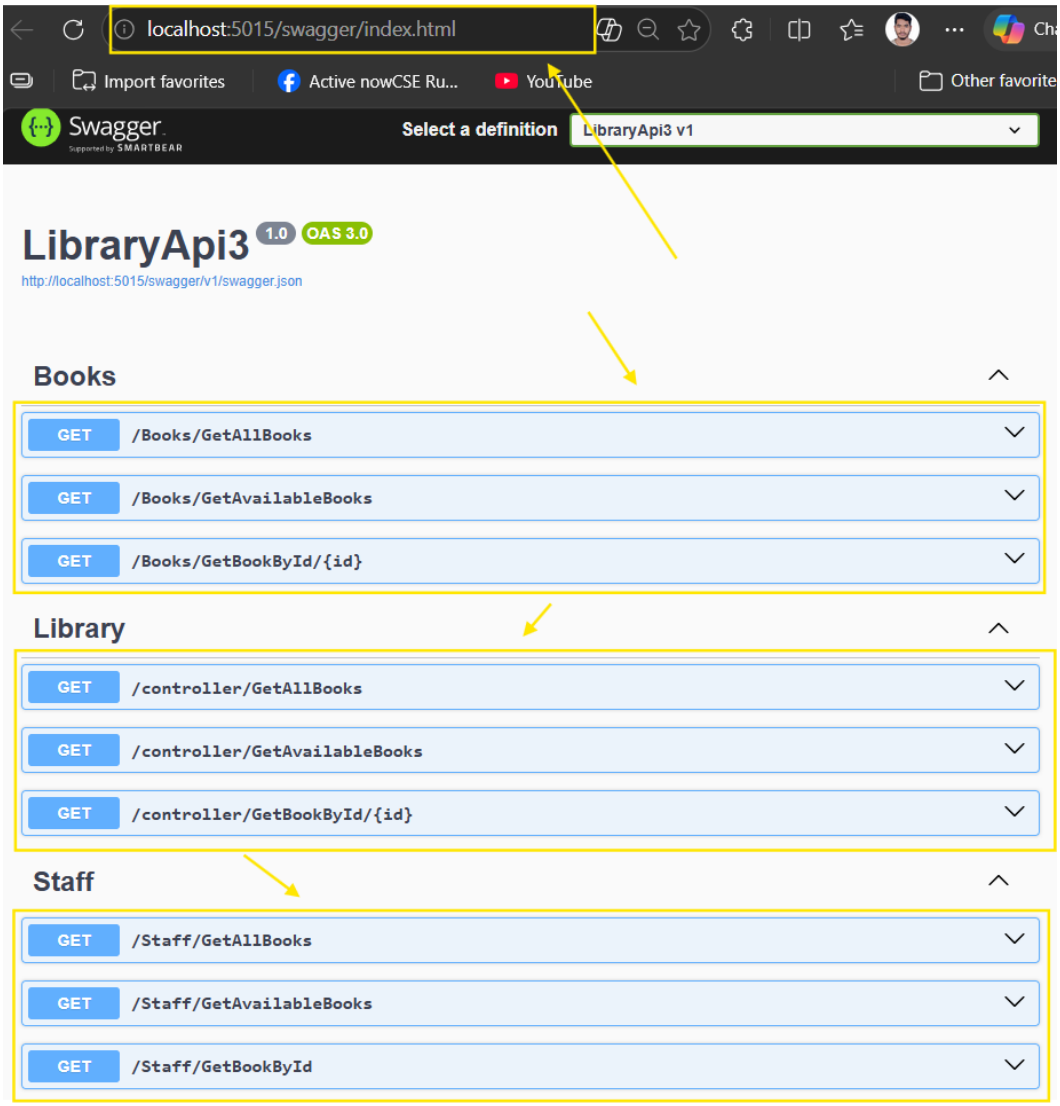
Command: *dotnet run* (we usually get the URL from here to see application response).

We can run the project and can directly navigate to application response or Swagger UI.

Use can run the command,

Command: *dotnet watch run*

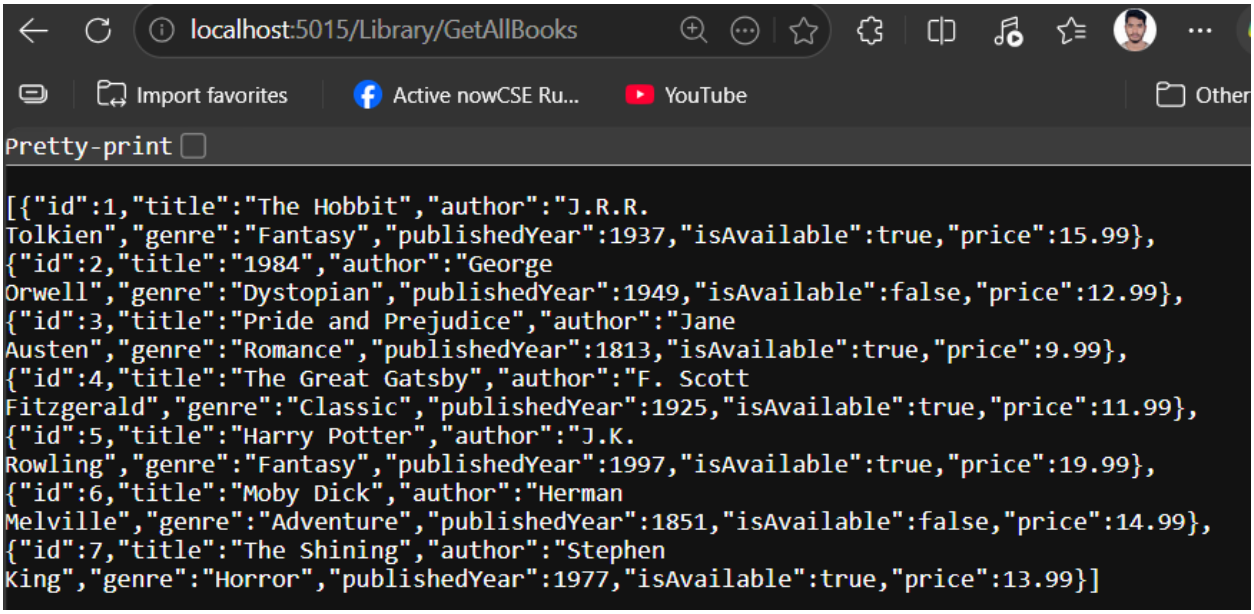
```
PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi3> dotnet watch run
dotnet watch 🔥 Hot reload enabled. For a list of supported edits, see https://aka.ms/dotnet/ho
t-reload.
  💡 Press "Ctrl + R" to restart.
dotnet watch 🔧 Building...
  Determining projects to restore...
  All projects are up-to-date for restore.
  LibraryApi3 -> E:\PracticeProjects\DotnetApi\Routing\LibraryApi3\bin\Debug\net8.0\LibraryApi
  3.dll
dotnet watch 🚀 Started
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5015
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: E:\PracticeProjects\DotnetApi\Routing\LibraryApi3
```


localhost:5015/swagger/index.html
Import favorites
Active nowCSE Ru...
YouTube
Other favorite
Swagger
Select a definition
LibraryApi3 v1
LibraryApi3
1.0
OAS 3.0
http://localhost:5015/swagger/v1/swagger.json
Books
GET /Books/GetAllBooks
GET /Books/GetAvailableBooks
GET /Books/GetBookById/{id}
Library
GET /controller/GetAllBooks
GET /controller/GetAvailableBooks
GET /controller/GetBookById/{id}
Staff
GET /Staff/GetAllBooks
GET /Staff/GetAvailableBooks
GET /Staff/GetBookById

TEST 1: URL using Token Replacement - /Library/GetAllBooks

Request: GET http://localhost:5015/Library/GetAllBooks

Response:

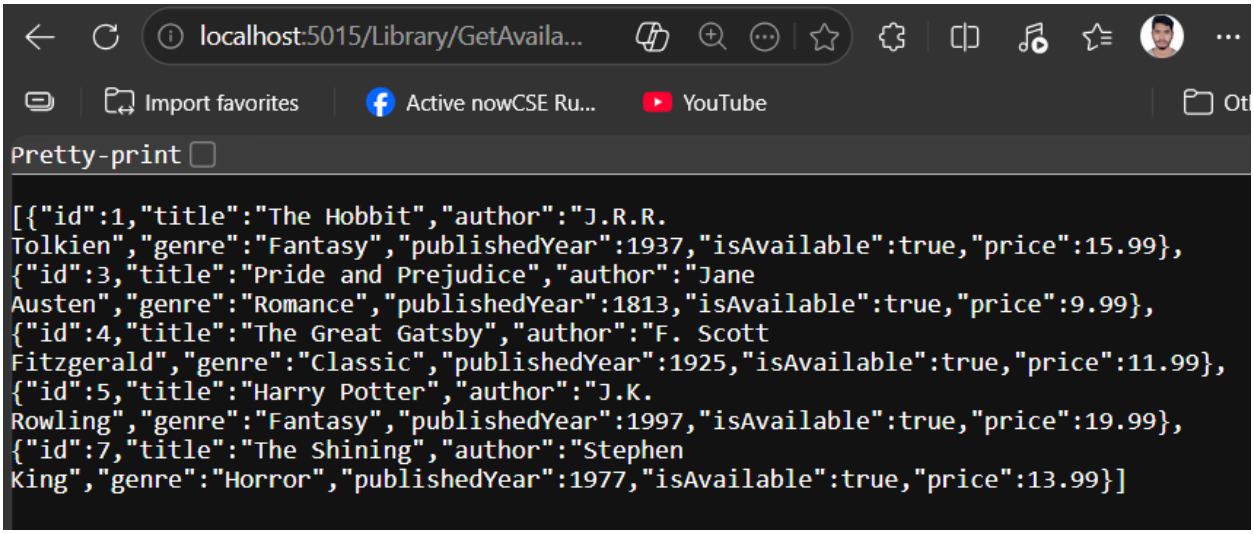



```
[{"id":1,"title":"The Hobbit","author":"J.R.R.
Tolkien","genre":"Fantasy","publishedYear":1937,"isAvailable":true,"price":15.99},
{"id":2,"title":"1984","author":"George
Orwell","genre":"Dystopian","publishedYear":1949,"isAvailable":false,"price":12.99},
{"id":3,"title":"Pride and Prejudice","author":"Jane
Austen","genre":"Romance","publishedYear":1813,"isAvailable":true,"price":9.99},
{"id":4,"title":"The Great Gatsby","author":"F. Scott
Fitzgerald","genre":"Classic","publishedYear":1925,"isAvailable":true,"price":11.99},
{"id":5,"title":"Harry Potter","author":"J.K.
Rowling","genre":"Fantasy","publishedYear":1997,"isAvailable":true,"price":19.99},
{"id":6,"title":"Moby Dick","author":"Herman
Melville","genre":"Adventure","publishedYear":1851,"isAvailable":false,"price":14.99},
{"id":7,"title":"The Shining","author":"Stephen
King","genre":"Horror","publishedYear":1977,"isAvailable":true,"price":13.99}]
```


TEST 2: URL with Action Token - /Library/GetAvailableBooks

Request: GET http://localhost:5015/Library/GetAvailableBooks

Response:

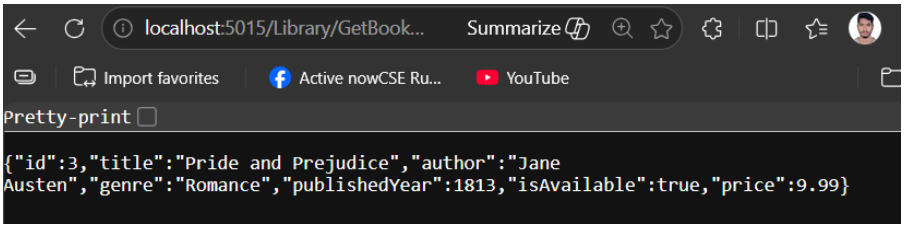



```
[{"id":1,"title":"The Hobbit","author":"J.R.R.
Tolkien","genre":"Fantasy","publishedYear":1937,"isAvailable":true,"price":15.99},
{"id":3,"title":"Pride and Prejudice","author":"Jane
Austen","genre":"Romance","publishedYear":1813,"isAvailable":true,"price":9.99},
{"id":4,"title":"The Great Gatsby","author":"F. Scott
Fitzgerald","genre":"Classic","publishedYear":1925,"isAvailable":true,"price":11.99},
{"id":5,"title":"Harry Potter","author":"J.K.
Rowling","genre":"Fantasy","publishedYear":1997,"isAvailable":true,"price":19.99},
{"id":7,"title":"The Shining","author":"Stephen
King","genre":"Horror","publishedYear":1977,"isAvailable":true,"price":13.99}]
```

TEST 3: URL with Token + Parameter - /Library/GetBookById/3

Request: GET http://localhost:5015/Library/GetBookById/3

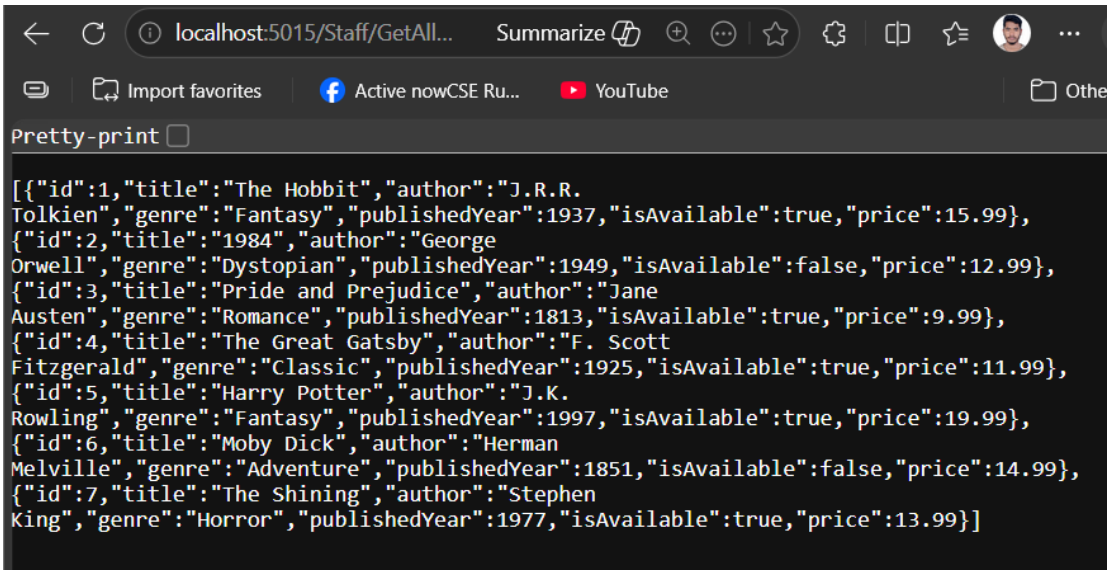
Response:



TEST 4: URL Alternative Controller (Staff) - /Staff/GetAllBooks

Request: GET http://localhost:5015/Staff/GetAllBooks

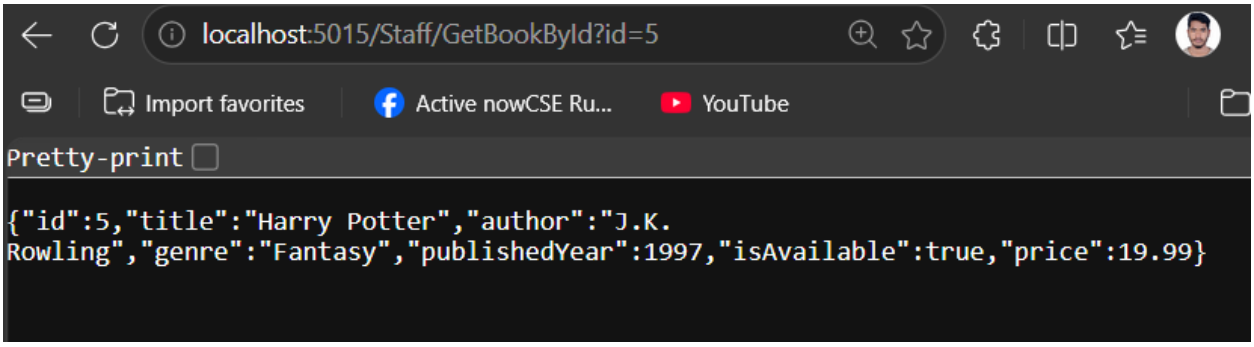
Response:



TEST 5: Staff Controller with Parameter - /Staff/GetBookById/5

Request: GET http://localhost:5015/Staff/GetBookById?=5

Response:



Key Takeaways

Concept	Explanation
[controller] token	Replaced with controller name (without "Controller")
[action] token	Replaced with action method name
Best Practice	Define [Route("[controller]/[action]")] at controller level
Renaming Safe	Change controller/action names → URLs update automatically
Combine with Parameters	[Route("[controller]/[action]/{id}")]